

Design a tiny expression language

Target language: arithmetic expressions with + and *, identifiers a, b, c, and parentheses.

Examples that must be accepted:

a + b

a + b * c

(a + b) * c

a * (b + c) + d

Part 1. LL-friendly grammar

1. Write a LL-style grammar that supports the language above
2. Write a program that parses such expressions using a top-down approach

The second task can be done in 2 ways:

1. Implement a recursive descent parser in their language of choice:

parse_expr, parse_term, parse_factor.

Input: token stream like [IDENT(a), '+', IDENT(b), '*', IDENT(c)].

Output: print a parenthesized structure, e.g.

(+ a (* b c)).

2. Use ANTLR tool to generate parser from grammar

Define expr, term, factor rules.

Generate parser (e.g. -Dlanguage=Python3).

Parse a + b * c and print tree.toStringTree(parser).

Part 2. LR-friendly grammar

1. Write an LR-style grammar that supports the language above
 2. Write a minimal Flex lexer that returns IDENT, '+', '*', '(', ')', and EOF.
 3. In Bison:
 - Use the grammar above.
 - In the Expr rules, add actions that print when a rule fires, e.g.:
 - a. Expr '+' Expr { printf("Reduce: E $\rightarrow$ E + E\n"); }
 4. Build and run on $a + b * c$ and $a * (b + c)$.
- Instead of 2-4 you can try to implement shift/reduce strategy on your own.